# Revolute Hackathon

Welcome!
08.01.2026

WIFI: Fab Hub Public + email address

Join the Discord!

# Registration

- **WIFI: Fab Hub Public + an email address**

**We're wearing aprons**

**Fab festival + vendor breakdown is now: There will be strangers walking through**

# Schedule

# Pt 0. Set up Hardware

Each team:

- Leader Arm
- Follower Arm
- Overhead Camera
- Wrist Camera
- Recomputer
- Monitor / keyboard / mouse
- Hot plate

Passwords

seeed or hacker123

Version A. Recompute Setup
Thanks to Yaohui from Seeedstudios!

This is not hackathon specific

https://wiki.seeedstudio.com/rebot_arm_b601_rs_lerobot/#calibrate-the-robotic-arm

This is hackathon specific

https://seeedstudio.feishu.cn/wiki/Bx6nwVV78i2ZdWkHfPVcCiJdnpe

# Part 1. Set up teleop

**Overview:**

- Robot follower = the big reBot arm
  - Talks on CANx
- Robot leader = the small FashionStar arm
  - Talks on ttyUSB0

Step 1. Find CAN port

Set 2. Run teleop command

Everything else should be set up already!

# Part 1. Set up teleop

## Detecting the PCAN-USB interface on Jetson

```
for i in /sys/class/net/can*; do

    [ "$(basename "$(readlink -f "$i/device/driver"
2>/dev/null)")" = "pcan" ] && basename "$i"

done
```

# Part 1. Set up teleop

## Detecting the PCAN-USB interface on Jetson

```
for i in /sys/class/net/can*; do

    [ "$(basename "$(readlink -f "$i/device/driver"
2>/dev/null)")" = "pcan" ] && basename "$i"

done
```

## Set up port variable

```
PCAN_IF=can1
```

# Part 1. Set up teleop

## Detecting the PCAN-USB interface on Jetson

```
for i in /sys/class/net/can*; do

    [ "$(basename "$(readlink -f "$i/device/driver" 2>/dev/null)")" = "pcan" ] && basename "$i"

done
```

## Set up port variable

```
PCAN_IF=can1
```

## Bring up CAN

```
sudo ip link set "$PCAN_IF" down 2>/dev/null

sudo ip link set "$PCAN_IF" type can bitrate 1000000 restart-ms 100

sudo ip link set "$PCAN_IF" txqueuelen 1000

sudo ip link set "$PCAN_IF" up
```

```
sudo add-apt-repository ppa:xtradeb/apps -y

sudo apt update

 sudo apt install chromium -y
```

# Part 1. Set up teleop

**Calibrate Follower Arm at zero position**

**lerobot-calibrate **

--robot.type=seeed_b601_rs_follower \

--robot.port="$PCAN_IF" \

--robot.id=follower1 \

--robot.can_adapter=socketcan

# Part 1. Set up teleop

**Calibrate Follower Arm at zero position**

```
lerobot-calibrate \
    --robot.type=seeed_b601_rs_follower \
    --robot.port="$PCAN_IF" \
    --robot.id=follower1 \
    --robot.can_adapter=socketcan
```

**DONT'T RUN THIS YET**

**Run Teleop**

```
lerobot-teleoperate \
    --robot.type=seeed_b601_rs_follower \
    --robot.port="$PCAN_IF" \
    --robot.id=follower1 \
    --robot.can_adapter=socketcan \
    --teleop.type=rebot_arm_102_leader \
    --teleop.port=/dev/ttyUSB0 \
    --teleop.id=rebot_arm_102_leader
```

# Part 1. Set up teleop

**SAFETY!**

**Trying lifting arm and dropping it -- it's very heavy!**

**Keep 1 meter away**

**DONT'T RUN THIS YET**

**Run Teleop**

```
lerobot-teleoperate \
    --robot.type=seeed_b601_rs_follower \
    --robot.port="$PCAN_IF" \
    --robot.id=follower1 \
    --robot.can_adapter=socketcan \
    --teleop.type=rebot_arm_102_leader \
    --teleop.port=/dev/ttyUSB0 \
    --teleop.id=rebot_arm_102_leader
```

```
snap changes                # find the stuck change's ID

sudo snap abort <id>
sudo systemctl restart snapd.socket

sudo systemctl restart snapd
```

# Pt 2. Record Data

Overview

1. Add cameras
2. Record locally

# Pt 2. Record Data

Add cameras

**lerobot-find-cameras opencv**

```
--- Detected Cameras ---
Camera #0:
  Name: OpenCV Camera @ 0
  Type: OpenCV
  Id: 0
  Backend api: AVFOUNDATION
  Default stream profile:
    Format: 16.0
    Width: 1920
    Height: 1080
    Fps: 15.0
--------------------
(more cameras ...)
```

# Pt 2. Record Data

Add cameras

**lerobot-find-cameras opencv**

Not sure which is which? Output Images at

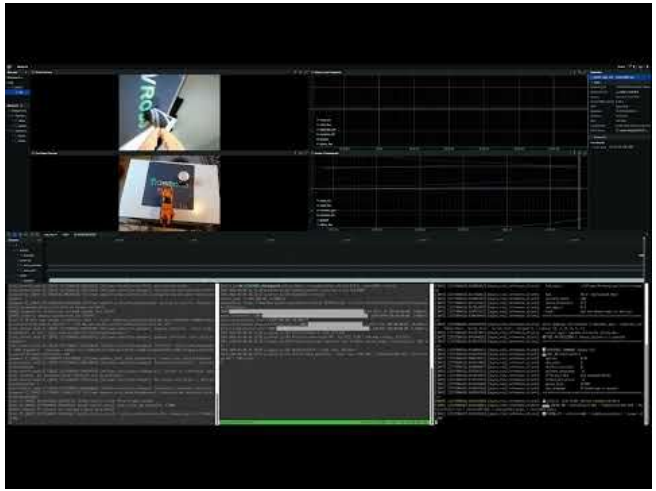
**~/lerobot/outputs/captured_images**

```
--- Detected Cameras ---
Camera #0:
  Name: OpenCV Camera @ 0
  Type: OpenCV
  Id: 0
  Backend api: AVFOUNDATION
  Default stream profile:
    Format: 16.0
    Width: 1920
    Height: 1080
    Fps: 15.0
--------------------
(more cameras ...)
```

# Pt2. Record data - teleop with cameras

```
lerobot-teleoperate \
    --robot.type=seeed_b601_rs_follower \
    --robot.port=can0 \
    --robot.id=follower1 \
    --robot.can_adapter=socketcan \
    --robot.cameras="{ front: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps:
30, fourcc: "MJPG"}, side: {type: opencv,
index_or_path: 2, width: 640, height: 480, fps:
30, fourcc: "MJPG"}}" \
    --teleop.type=rebot_arm_102_leader \
    --teleop.port=/dev/ttyUSB0 \
    --teleop.id=rebot_arm_102_leader \
    --display_data=true
```

**Should pop up rerun screen**



# Pt 3. Record data

**Make sure background is clean! Without the other group members**

```
lerobot-record \
   --robot.type=seeed_b601_rs_follower \
   --robot.port=$PCAN_IF \
   --robot.id=follower1 \
   --robot.can_adapter=socketcan \
   --robot.cameras="{ front: {type: opencv, index_or_path:
0, width: 640, height: 480, fps: 30, fourcc: "MJPG"}, side:
{type: opencv, index_or_path: 2, width: 640, height: 480,
fps: 30, fourcc: "MJPG"}}" \
   --teleop.type=rebot_arm_102_leader \
   --teleop.port=/dev/ttyUSB0 \
   --teleop.id=rebot_arm_102_leader \
   --display_data=true \
 --dataset.repo_id=seeed_rebot_b601_rs/test \
   --dataset.num_episodes=1 \
   --dataset.single_task="Grab the black cube" \
   --dataset.push_to_hub=false \
   --dataset.episode_time_s=10 \
   --dataset.reset_time_s=5
```

# Pt 3. Record data -> Train local

**Make sure background is clean! Without the other group members**

```
lerobot-record \
   --robot.type=seeed_b601_rs_follower \
   --robot.port=$PCAN_IF \
   --robot.id=follower1 \
   --robot.can_adapter=socketcan \
   --robot.cameras="{ front: {type: opencv, index_or_path:
0, width: 640, height: 480, fps: 30, fourcc: "MJPG"}, side:
{type: opencv, index_or_path: 2, width: 640, height: 480,
fps: 30, fourcc: "MJPG"}}" \
   --teleop.type=rebot_arm_102_leader \
   --teleop.port=/dev/ttyUSB0 \
   --teleop.id=rebot_arm_102_leader \
   --display_data=true \
 --dataset.repo_id=seeed_rebot_b601_rs/test \
   --dataset.num_episodes=1 \
   --dataset.single_task="Grab the black cube" \
   --dataset.push_to_hub=false \
   --dataset.episode_time_s=10 \
   --dataset.reset_time_s=5
```

**Make sure the `repo_id` matches the name used during data collection and add `--push_to_hub=false`.**

```
lerobot-train \
  --dataset.repo_id=seeed_rebot_b601_rs/test \
  --policy.type=act \
  --output_dir=outputs/train/act_rebot_test \
  --job_name=act_rebot_test \
  --policy.device=cuda \
  --policy.use_amp=true \
  --batch_size=1 \
  --num_workers=0 \
  --wandb.enable=false \
  --push_to_hub=false \
  --steps=1
```

**Delete directory or rename if already exists**

# Pt 4. Inference Local

1. Check recorded video looks good

e.g.

**/home/seeed/outputs/train/test/checkpoints/000001/pretrained_model**

Look for outputs:

- config.json
- model.safetensors
- policy_postprocessor.json
- policy_postprocessor_step... (truncated name)
- policy_preprocessor.json
- policy_preprocessor_step... (truncated name)
- train_config.json

# Pt 4. Inference Local

Use a policy checkpoint as input  to lerobot-record.

```
lerobot-record \
    --robot.type=seeed_b601_rs_follower \
    --robot.port=$PCAN_IF \
    --robot.id=follower1 \
    --robot.can_adapter=socketcan \
    --robot.cameras="{ front: {type: opencv, index_or_path:
0, width: 640, height: 480, fps: 30, fourcc: "MJPG"}, side:
{type: opencv, index_or_path: 2, width: 640, height: 480,
fps: 30, fourcc: "MJPG"}}" \
    --teleop.type=rebot_arm_102_leader \
    --teleop.port=/dev/ttyUSB0 \
    --teleop.id=rebot_arm_102_leader \
    --display_data=true \
  --dataset.repo_id=seeed_rebot_b601_rs/test \
    --dataset.num_episodes=1 \
    --dataset.single_task="Grab the black cube" \
    --dataset.push_to_hub=false \
    --dataset.episode_time_s=10 \
    --dataset.reset_time_s=5
```

```
lerobot-record \
  --robot.type=seeed_b601_rs_follower \
  --robot.port= $PORT_IF \
  --robot.can_adapter=socketcan \
  --robot.cameras='{ front: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30,
fourcc: "MJPG"}, side: {type: opencv,
index_or_path: 2, width: 640, height: 480, fps: 30,
fourcc: "MJPG"} }' \
  --robot.id=follower1 \
  --display_data=false \
  --dataset.repo_id= seeed_rebot_b601_rs/test \
  --dataset.single_task=" Grab the black cube" \
  --policy.path=outputs/train/act_rebot_test/checkp
  oints/last/pretrained_model
```

# From docs @ feishu

### How should I position the cameras?

Use two complementary camera views:

- **Overhead camera:** Position it above the workspace so that it captures the entire robotic arm, target objects, placement area, and a clean working surface.
- **Wrist camera:** Mount it securely on the robot's end effector so that the gripper, target object, and grasping area remain visible during close-range operations.

Before collecting data, verify that the wrist camera and its cable do not interfere with joint movement or collide with the robot.

### Can I move the cameras after collecting the dataset?

No. The camera position, angle, orientation, resolution, and field of view used during data collection must remain strictly consistent with the final deployment setup.
Moving or rotating a camera after training may significantly reduce performance and can make the collected dataset unusable.

# From docs @ feishu

### What environmental conditions should remain stable?

https://huggingface.co/blog/lerobot-datasets#what-makes-a-good-dataset
Keep the following conditions as consistent as possible:

- Camera position and angle
- Ambient lighting
- Robot base and workspace position
- Background appearance
- Image resolution and camera order

Avoid unstable backgrounds, moving pedestrians, strong reflections, changing shadows, and frequent lighting changes. Excessive environmental variation may cause the robot to identify or grasp objects incorrectly.

### How should I record each episode?

Each episode should:

- Start from a clear and reproducible initial state.
- Contain one complete pick-and-place operation.
- Use smooth and continuous demonstrations.
- End only after the object has been placed successfully and the scene has remained stable briefly.

Remove episodes containing failed grasps, collisions, camera freezes, severe image blur, or incorrect actions.

# From docs @ feishu

### How many episodes should I collect?

For pick-and-place tasks, collect approximately **5–10 successful episodes around each target position**.
Vary the object position and orientation within the expected deployment range, while keeping the camera setup and operating strategy consistent. More high-quality episodes generally improve performance, but data quality and distribution are more important than simply increasing the total number.

### What should I check before training?

[ ] The overhead camera covers the complete workspace

[ ] The wrist camera clearly captures the gripper and target object

[ ] Camera positions match the final deployment setup

[ ] Lighting and background remain stable

[ ] Camera names and order are correct

[ ] Images and robot actions are synchronized

[ ] Every retained episode completes the task successfully

# From docs @ feishu

**Key principle:** Keep the cameras and deployment environment consistent, collect smooth and successful demonstrations, and ensure that every expected target position is sufficiently represented.

Version B. Autodesk
… After lunch